You can even bring methods into an individual object:

09-configuring-rspec/03/rspec_configure.rb
```
average_person = AveragePerson.new
average_person.extend Singing
```

RSpec provides the same kind of interface inside RSpec.configure blocks. By calling include, prepend, or extend on the config object, you can bring extra methods into your examples or groups.

09-configuring-rspec/03/rspec_configure.rb
```
RSpec.configure do |config|
  # Brings methods into each example
  config.include ExtraExampleMethods

  # Brings methods into each example,
  # overriding methods with the same name
  # (rarely used)
  config.prepend ImportantExampleMethods

  # Brings methods into each group (alongside let/describe/etc.)
  # Useful for adding to RSpec's domain-specific language
  config.extend ExtraGroupMethods
end
```

Because they work like their Ruby counterparts that you already use, these three config methods are great for sharing Ruby methods across your specs. If you need to share more, though—such as hooks or let definitions—you'll need to define a shared example group. You can then bring in this shared group automatically inside your configure block:

09-configuring-rspec/03/rspec_configure.rb
```
RSpec.configure do |config|
  config.include_context 'My Shared Group'
end
```

Now that we've covered sharing code via a configure block, let's talk about controlling how RSpec runs.

Filtering

Several times throughout this book, you've found the need to run just some of the examples in your suite. At various points, you've used RSpec's filtering to run the following subsets of specs:

- A single example or group by name
- Only the specs matching a certain piece of metadata, such as :fast
- Just the examples you're focusing your attention on
- Only the examples that failed the last time they ran